



# Planificación

Arquitectura y Sistemas Operativos

Unidad 3



# Planificación

- Un sistema operativo debe asignar los recursos de la computadora entre las necesidades potencialmente competitivas de múltiples procesos
- En el caso del procesador, el recurso que se debe asignar es el tiempo de ejecución en el procesador
  - La forma de asignarlo es la planificación
- La función de planificación debe estar diseñada para satisfacer varios objetivos que incluyen:
  - Equidad
  - Ausencia de inanición de cualquier proceso
  - Uso eficiente del tiempo del procesador
  - Baja sobrecarga



# Objetivos de la planificación

---

- Asignar procesos a ejecutar por el/los procesador/es
- Tiempo de respuesta
- Rendimiento del sistema
- Rendimiento del procesador



# Tipos de planificación

- En muchos sistemas, esta actividad de planificación se divide en tres funciones independientes:

|                             |                                                                                                                    |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------|
| Planificación a largo plazo | La decisión de añadir un proceso al conjunto de procesos a ser ejecutados                                          |
| Planificación a medio plazo | La decisión de añadir un proceso al número de procesos que están parcialmente o totalmente en la memoria principal |
| Planificación a corto plazo | La decisión por la que un proceso disponible será ejecutado por el procesador                                      |



# Planificación del procesador (*processor scheduling*)

---

- En un sistema de un solo procesador, sólo se puede ejecutar un proceso a la vez. Otros deben esperar hasta que la CPU esté libre y se pueda reprogramar
- El objetivo de la multiprogramación es tener algún proceso en marcha en todo momento, para maximizar la utilización de la CPU
- Un proceso se ejecuta hasta que termina o debe esperar, típicamente, por la terminación de una solicitud de E/S



# Planificación del procesador (*processor scheduling*)

- Un proceso se ejecuta hasta que termina o debe esperar, típicamente, por la terminación de una solicitud de E/S
  - En un sistema informático simple, la CPU queda inactiva. Todo este tiempo de espera se desperdicia; no se realiza ningún trabajo útil
  - Con la multiprogramación, se trata de utilizar este tiempo de manera productiva. Varios procesos se mantienen en la memoria a la vez. Cuando un proceso tiene que esperar, el SO toma la CPU, saca ese proceso y da la CPU a otro proceso. Cada vez que un proceso tiene que esperar, otro proceso puede apoderarse del uso de la CPU
- El sistema operativo decide qué proceso ocupa el procesador cuando éste queda libre
- El SO también decide en qué momento el proceso que está en ejecución debe abandonar el procesador



# Modos de decisión

Las decisiones de planificación de la CPU pueden tener lugar bajo las siguientes cuatro circunstancias:

1. Cuando un proceso cambia del estado de **ejecución** al estado de **espera** (por ejemplo, como resultado de una petición de E/S o una invocación de `wait()` para la terminación de un proceso hijo)
  2. Cuando un proceso cambia del estado de **ejecución** al estado **preparado** (por ejemplo, cuando se produce una interrupción)
  3. Cuando un proceso cambia del estado de **espera** al estado **preparado** (por ejemplo, al finalizar la llamada de E/S)
  4. Cuando un proceso termina
- Para las situaciones 1 y 4, no hay nada para planificar. Se debe tomar un nuevo proceso para su ejecución
  - Hay una opción, sin embargo, para las situaciones 2 y 3



# Modos de decisión

- No expropiativo (*non preemptive*). Cooperativo
  - Una vez que un proceso está en el estado de ejecución, continuará hasta que termina o se bloquea para E/S
  - **Riesgo de acaparamiento injusto de la CPU**
  - Windows 3.11, Apple Macintosh (original)
- Expropiativo (*preemptive*)
  - El proceso actualmente en ejecución puede ser interrumpido y movido al estado Listo por el SO
  - Permite un mejor servicio ya que ningún proceso puede monopolizar el procesador durante mucho tiempo
  - Para implementar tiempo compartido y tiempo real, es necesaria una planificación expulsiva o expropiativa
  - **Riesgo en la consistencia de datos compartidos**
  - Unix, Windows, Mac OS X ...





# Política de planificación

- ¿Cuál es la mejor decisión a tomar para que un proceso entre en la CPU?
  - ¿En orden de llegada?
  - ¿Primero la tarea más corta?
  - ¿Por prioridades?
- Se deben definir posibles criterios de valoración de las políticas a aplicar
- Diferentes algoritmos de planificación de CPU tendrán propiedades diferentes
  - La elección de un algoritmo particular puede favorecer una clase de procesos sobre otra
- Al elegir que algoritmo utilizar en una situación particular, se debe considerar las propiedades de los diversos algoritmos
- No existe una política de planificación óptima para todos los criterios
  - Habrá que llegar a un compromiso



# Criterios de planificación

Distintos criterios para comparar algoritmos de planificación de CPU

- **Utilización de la CPU:** % de tiempo de ocupación de la CPU
- **Rendimiento:** Cantidad de procesos que se completan por unidad de tiempo
- **Tiempo de retorno:** Intervalo entre el tiempo de presentación de un proceso y el tiempo su finalización
- **Tiempo de espera:** La suma de los períodos de espera en la cola lista
- **Tiempo de respuesta:** Tiempo desde la presentación de una solicitud hasta que se produce la primera respuesta



# *FCFS: First-Come-First-Served* (en orden de llegada)

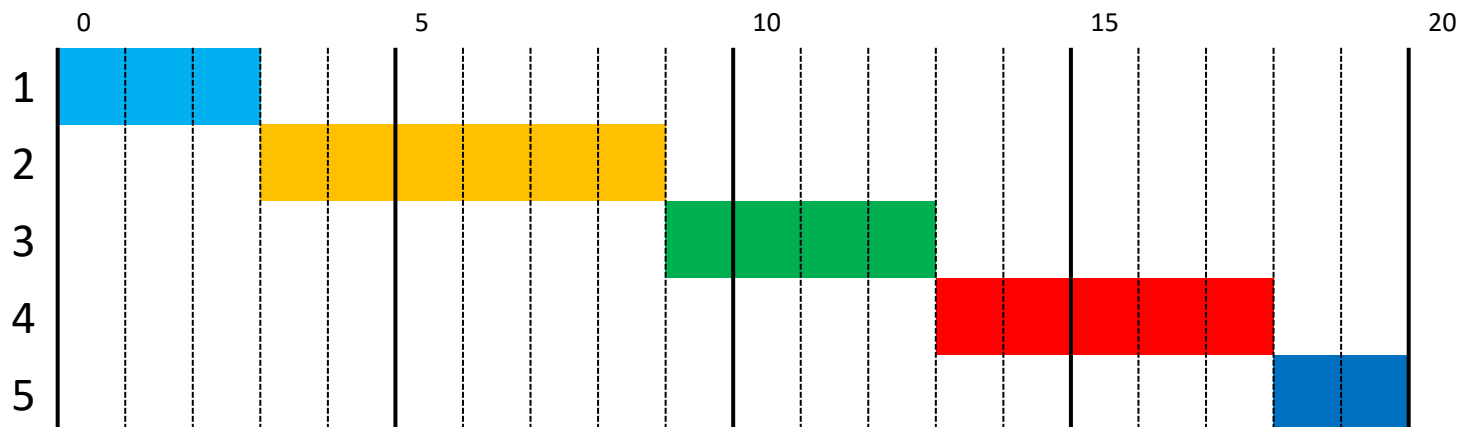
- La cola de preparados se gestiona como una FIFO
  - Cada proceso se une a la cola Listo
  - Cuando el proceso actual deja de ejecutarse, se selecciona el proceso más antiguo en la cola Listo
- Simple de implementar
- Muy sensible al orden de llegada de los procesos
  - Un proceso corto puede tener que esperar un tiempo muy largo antes de que se pueda ejecutar
- Favorece los procesos vinculados a la CPU
- Perjudica a los procesos intensivos en E/S



# *FCFS: First-Come-First-Served*

(en orden de llegada)

| Proceso | Tiempo arribo | duración |
|---------|---------------|----------|
| P1      | 0             | 3        |
| P2      | 2             | 6        |
| P3      | 4             | 4        |
| P4      | 6             | 5        |
| P5      | 8             | 2        |

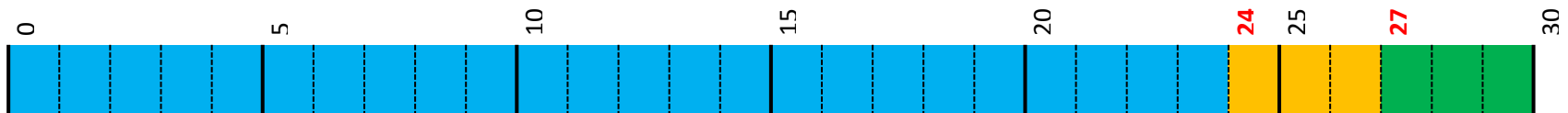




# FCFS: First-Come-First-Served (en orden de llegada)

| Proceso | Duración |
|---------|----------|
| P1      | 24       |
| P2      | 3        |
| P3      | 3        |

CASO 1: En t=0; orden de llegada P1-P2-P3



|                   | P1 | P2 | P3 |
|-------------------|----|----|----|
| Tiempo de espera  | 0  | 24 | 27 |
| Tiempo de retorno | 24 | 27 | 30 |

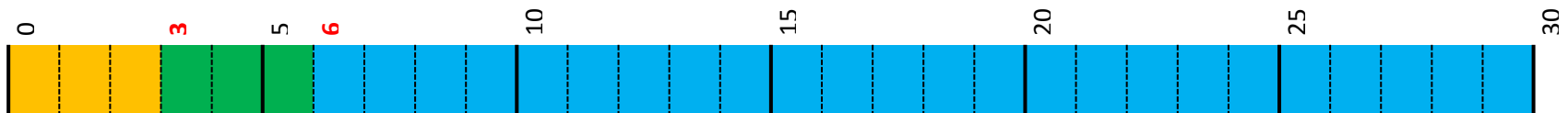
Tiempo de espera medio  $\bar{t}_e = \frac{t_{e1} + t_{e2} + t_{e3}}{3} = \frac{0 + 24 + 27}{3} = 17$



# FCFS: First-Come-First-Served (en orden de llegada)

| Proceso | Duración |
|---------|----------|
| P1      | 24       |
| P2      | 3        |
| P3      | 3        |

CASO 2: En  $t=0$ ; orden de llegada P2-P3-P1



|                   | P1 | P2 | P3 |
|-------------------|----|----|----|
| Tiempo de espera  | 6  | 0  | 3  |
| Tiempo de retorno | 30 | 3  | 6  |

Tiempo de espera medio  $\bar{t}_e = \frac{t_{e1} + t_{e2} + t_{e3}}{3} = \frac{6 + 0 + 3}{3} = 3$



# *SPN: Shortest Process Next* (el mas corto es el siguiente)

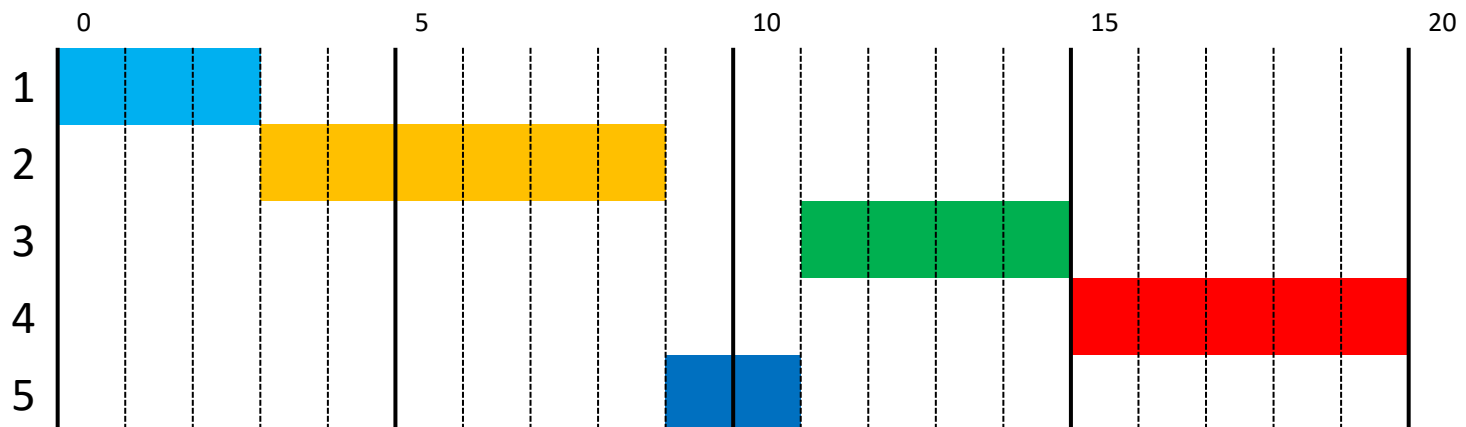
- $SPN = SJF$  (*Shortest Job First*)
- Política no expropiativa
- Se selecciona el proceso con el tiempo de procesamiento más corto esperado
- El proceso corto salta por delante de procesos más largos
- Minimiza el tiempo de espera medio (óptimo)
- Riesgo de inanición de los procesos con ráfagas de larga duración
- No se puede predecir la siguiente ráfaga
  - Una implementación real debe calcular una estimación



# *SPN: Shortest Process Next*

(el mas corto es el siguiente)

| Proceso | Tiempo arribo | duración |
|---------|---------------|----------|
| P1      | 0             | 3        |
| P2      | 2             | 6        |
| P3      | 4             | 4        |
| P4      | 6             | 5        |
| P5      | 8             | 2        |

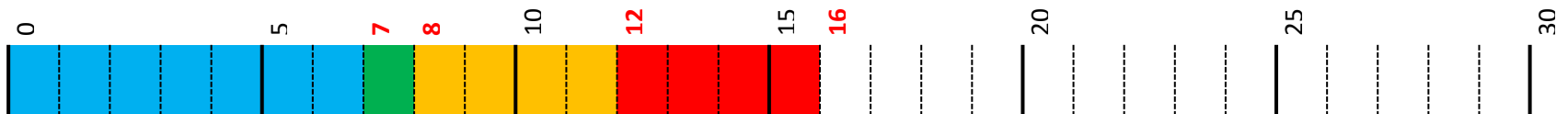






# SPN: Shortest Process Next (el mas corto es el siguiente)

| Proceso | Llegada | Duración |
|---------|---------|----------|
| P1      | 0       | 7        |
| P2      | 2       | 4        |
| P3      | 4       | 1        |
| P4      | 5       | 4        |



P1      P2      P3      P4

Tiempo de espera      0      8-2      7-4      12-5

Tiempo de retorno      7      12-2      8-4      16-5

Tiempo de espera medio 
$$\bar{t}_e = \frac{t_{e1} + t_{e2} + t_{e3} + t_{e4}}{4} = \frac{0 + 6 + 3 + 7}{4} = 4$$



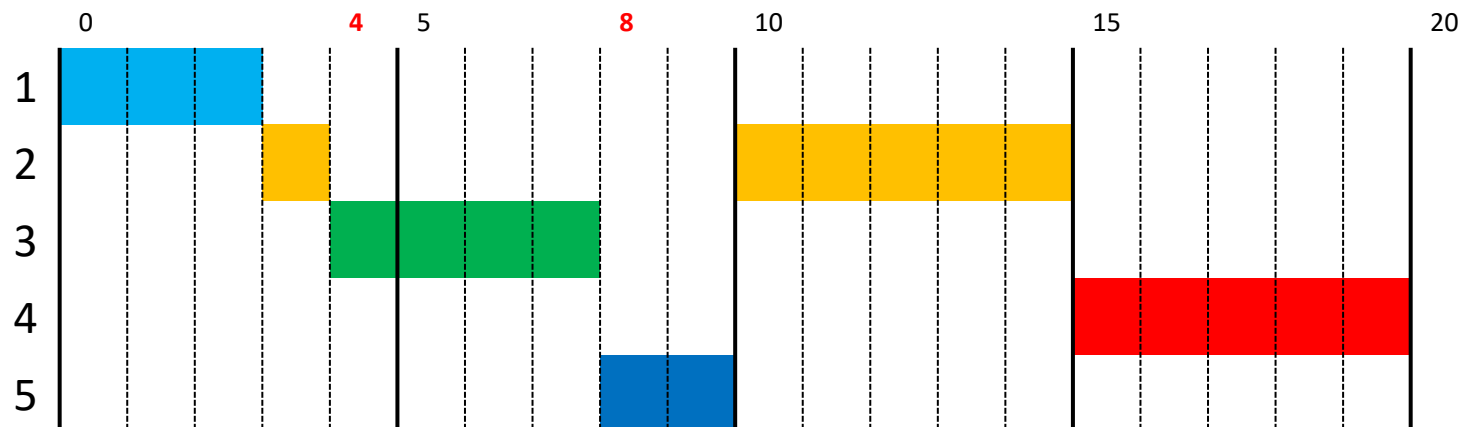
# *SRT: Shortest Remaining Time*

- Planificación con selección del proceso con tiempo restante más corto
- Versión expropiativa del proceso más corto de la política SPN
- El proceso en CPU es desalojado si llega a la cola un proceso con duración más corta
- Debe estimar el tiempo de procesamiento



# *SRT: Shortest Remaining Time*

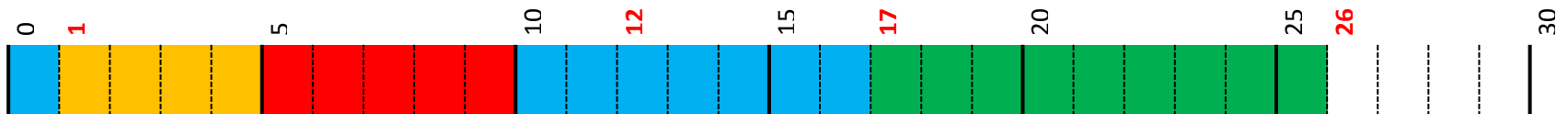
| Proceso | Tiempo arribo | duración |
|---------|---------------|----------|
| P1      | 0             | 3        |
| P2      | 2             | 6        |
| P3      | 4             | 4        |
| P4      | 6             | 5        |
| P5      | 8             | 2        |





# *SRT: Shortest Remaining Time*

| Proceso | Llegada | Duración |
|---------|---------|----------|
| P1      | 0       | 8        |
| P2      | 1       | 4        |
| P3      | 2       | 9        |
| P4      | 3       | 5        |



P1      P2      P3      P4

Tiempo de espera      10-1      2-2      17-2      5-3

Tiempo de retorno      17-0      5-1      26-2      10-3

Tiempo de espera medio  $\bar{t}_e = \frac{t_{e1} + t_{e2} + t_{e3} + t_{e4}}{4} = \frac{9 + 0 + 15 + 2}{4} = 6.5$

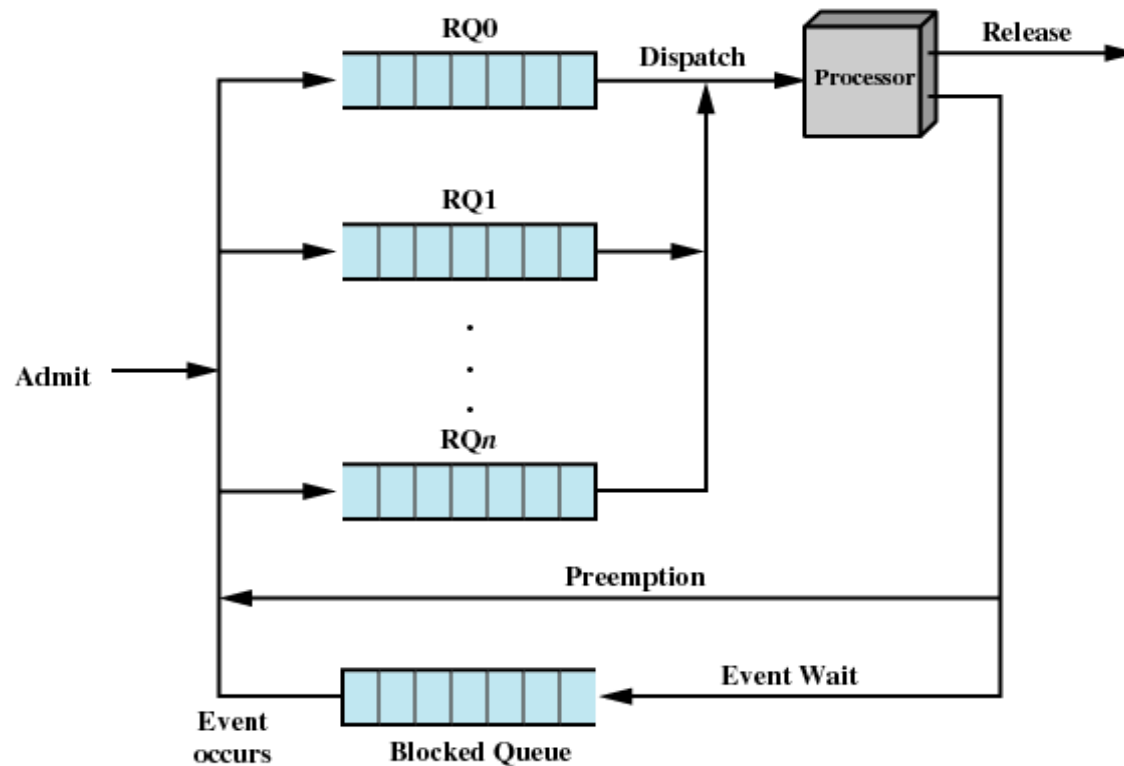


# Planificación basada en prioridades

- Cada proceso tiene una prioridad asignada
- Toma la CPU aquel proceso en estado listo con mayor prioridad
- La política puede ser expropiativa o no
- Prioridades definidas:
  - De forma interna (por el SO)
  - Externa (definida por los usuarios)
  - El SJF/SPN es un ejemplo (prioridad = duración estimada)
- Riesgo de inanición de los procesos con menor prioridad
  - Solución: envejecimiento (aging). Aumentar progresivamente la prioridad a los procesos en espera



# Planificación basada en prioridades





# *Round Robin* (turno rotativo)

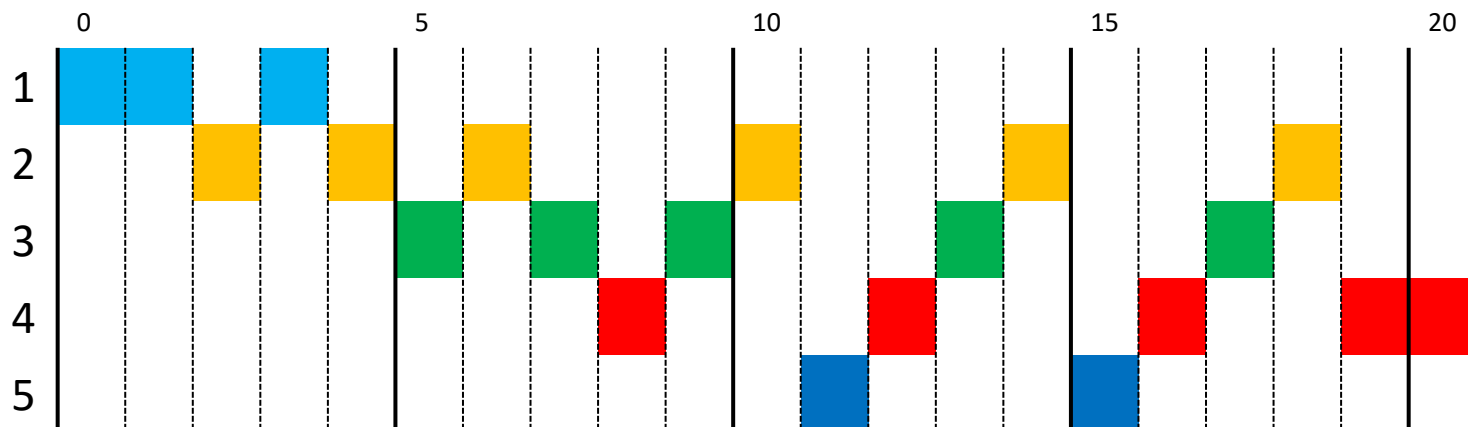
- También conocido como planificación cíclica
- Adecuado para implementar sistemas de tiempo compartido
- Similar a FCFS, pero cada proceso dispone de una “ventana” (*quantum*) de tiempo de duración  $Q$  para estar en la CPU
- Si transcurre  $Q$  y el proceso no liberó la CPU, el planificador lo desaloja y lo ingresa al final de la cola de listos
- La cola de listos se gestiona como FIFO
- Típicamente  $Q$  está entre 10 y 200 ms
- Si hay  $N$  procesos en cola, el peor tiempo de respuesta es:  
 $Q \cdot (N-1)$



# Round Robin

| Proceso | Tiempo arribo | duración |
|---------|---------------|----------|
| P1      | 0             | 3        |
| P2      | 2             | 6        |
| P3      | 4             | 4        |
| P4      | 6             | 5        |
| P5      | 8             | 2        |

$Q=1$





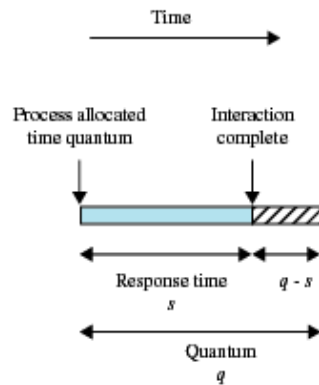


# *Round Robin: Influencia de $Q$*

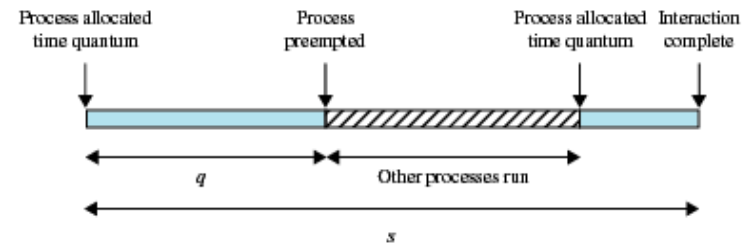
- Si  $Q$  es muy grande, los procesos terminan sus ráfagas de CPU antes que termine su ventana temporal
  - se comporta como un FCFS
- Si  $Q$  es muy pequeño, tiende a un sistema en el que cada proceso dispone de un procesador a  $1/N$  de la velocidad del procesador real (procesador compartido)
- Si  $Q$  es muy pequeño, ocurren más cambios de contexto y baja el rendimiento ( $Q$  debería ser mucho mayor que el tiempo que dura un cambio de contexto)



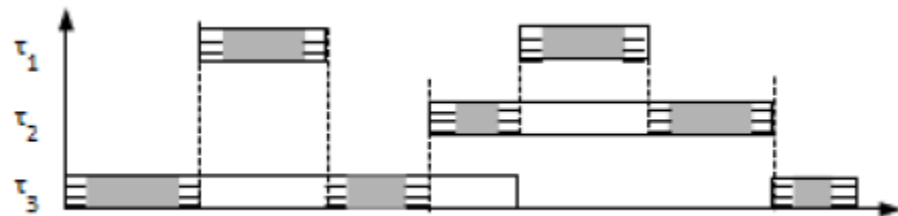
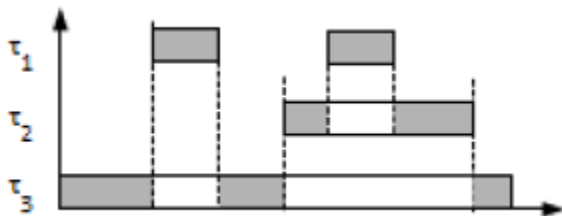
# Round Robin: Influencia de $Q$



(a) Time quantum greater than typical interaction

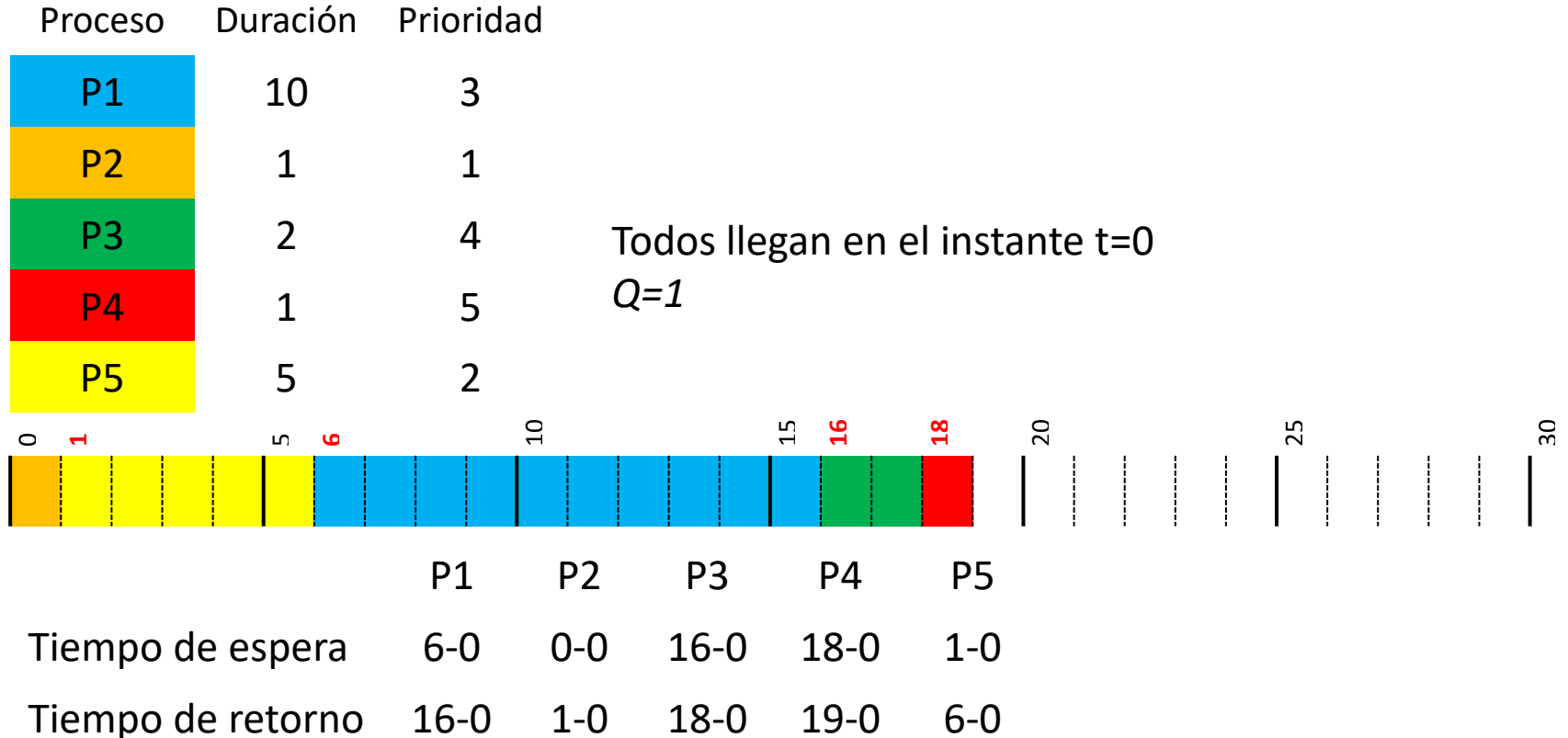


(b) Time quantum less than typical interaction





# Round Robin



Tiempo de espera medio  $\bar{t}_e = \frac{t_{e1} + t_{e2} + t_{e3} + t_{e4} + t_{e5}}{5} = \frac{6 + 0 + 16 + 18 + 1}{5} = 8.2$



# Planificación de multiprocesadores

- Cuando un sistema informático tiene más de un procesador, el diseño de la planificación plantea nuevas cuestiones
- Se puede clasificar los sistemas multiprocesador como:
  - **Débilmente acoplado o multiprocesador distribuido, o cluster:** Formado por sistemas relativamente autónomos; cada procesador tiene su propia memoria principal y canales de E/S
  - **Procesadores de funcionalidad especializada:** Hay un procesador de propósito general maestro y procesadores especializados que son controlados por el procesador maestro al cual le proporcionan servicios
  - **Procesamiento fuertemente acoplado:** Formado por un conjunto de procesadores que comparten la memoria principal y están bajo el control integrado de un único sistema operativo



# Planificación de multiprocesadores

- Procesamiento asimétrico
  - Un procesador se encarga de planificar el trabajo de los restantes (técnica en desuso)
- Procesamiento simétrico
  - Cada procesador realiza su planificación
  - Cola única de procesos preparados en memoria compartida
  - Una cola por procesador
  - Se debe asegurar que dos procesadores independientes no elijan planificar el mismo proceso y que los procesos no se pierdan de la cola



# Política para planificación de multiprocesadores

- Afinidad al procesador
  - Mantener al proceso siempre en un mismo procesador, para que aproveche los datos que puedan estar en la caché de la CPU
- Equilibrio de carga
  - Tratar de que no haya procesadores muy ocupados o muy ociosos
  - Migración comandada (*push*)
    - Una tarea supervisora comprueba la carga y traslada procesos
  - Migración solicitada (*pull*)
    - Un procesador ocioso puede “robar” procesos a otro más cargado



# Bibliografía complementaria

---

- Sistemas Operativos: aspectos internos y principios de diseño (Stallings, 2005, 5ª edición)
  - Capítulos 9, 10
- Fundamentos de los Sistemas Operativos (Silberschatz, Galvin, Gagne. 2006, 7ª edición)
  - Capítulo 6